Doc 2 — Modelo de datos MongoDB (TP #2 · «Jugátelo»)

Adaptado del enunciado base `2_Modelo_de_datos_MongoDB.pdf` (v1.0, 26/10/2026) a la temática de juegos de mesa. Colecciones `usuarios`, `publicaciones`, `comentarios`: esquemas Mongoose, índices, relaciones, ejemplos y consultas/aggregations de los Sprints 1 a 4. Ver también 1_analisis_funcional.md y 3_roadmap_por_sprints.md.

1. Convenciones

- Tres colecciones: **usuarios**, **publicaciones** (juegos de mesa y juntadas) y **comentarios**. Modelos Mongoose: `Usuario`, `Publicacion`, `Comentario` (decorador `@Schema` de `@nestjs/mongoose`).
- Identificadores en camelCase, sin tildes ni ñ (`contrasena`, `descripcion`, `publicacion`).
- `timestamps: true` en todas: Mongoose agrega `createdAt` y `updatedAt`. `createdAt` es la «fecha» de publicaciones y comentarios y la base de las estadísticas (S-23).
- Bajas lógicas con booleanos: `usuarios.habilitado` y `publicaciones.activa`. Nada se borra; las consultas filtran por esos campos.
- Imágenes en Cloudinary: se guarda `secure_url` y `public_id` (para poder borrar la imagen si falla el alta).
- Fechas en UTC; las estadísticas agrupan en hora de Argentina (S-18).
- S-01/S-03/S-27: cada documento de `publicaciones` es un juego de mesa (`tipo = 'juego'`) o una juntada (`tipo = 'juntada'`); una sola imagen opcional (si fueran varias: `imagenes: [{ url, publicId }]`). Los campos propios de cada tipo son opcionales en el esquema y se exigen según el tipo en el DTO (RN-14).

2. Relaciones

```
usuarios 1 ──< publicaciones        publicaciones.usuario  -> usuarios._id   (N:1, autor)
usuarios M ──< publicaciones        publicaciones.meGusta[] -> usuarios._id  (N:M, embebido)
publicaciones 1 ──< comentarios     comentarios.publicacion -> publicaciones._id (N:1, indexada)
usuarios 1 ──< comentarios          comentarios.usuario     -> usuarios._id  (N:1, autor)
```

3. Embebido vs. referencia

Dato	Decisión	Justificación
Comentarios	Colección propia con referencia a publicación y usuario	Se paginan (offset/limit, más recientes primero), se cuentan por rango de fechas y por publicación (S4) y se editan individualmente. Embebidos, la publicación crecería sin límite (tope 16 MB), la paginación requeriría `$slice` y las estadísticas obligarían a `$unwind` de todas las publicaciones.
Me gusta	**Embebido**: array `meGusta` de ObjectId + contador `cantidadMeGusta`	Solo se necesita saber si un usuario ya dio me gusta y cuántos hay. `$addToSet` evita duplicados y `$pull` lo quita, ambos atómicos sobre un único documento. El contador permite ordenar con un índice `{ cantidadMeGusta: -1 }` sin calcular `$size`. Volumen acotado en el TP.

Dato	Decisión	Justificación
Autor de publicación/comentario	Referencia a usuarios	El nombreUsuario y la foto pueden cambiar; con referencia se muestran siempre actualizados (populate o `$lookup`). Además se filtra por usuario (usuarioId) y se agrupa por usuario (estadísticas).
Imagen	Campos simples (URL + public_id)	El binario vive en Cloudinary.

4. Colección `usuarios`

4.1 Esquema

Campo	Tipo	Req.	Default	Validación / notas	Sprint
_id	ObjectId	sí	auto	Es el «id» de la payload JWT	S1
nombre	String	sí	—	trim, 2–50, solo letras y espacios	S1
apellido	String	sí	—	trim, 2–50, solo letras y espacios	S1
correo	String	sí	—	trim, lowercase, email, **único** (S-22)	S1
nombreUsuario	String	sí	—	trim, 3–20, `[a-zA-Z0-9._]`, **único** sin distinguir mayúsculas (índice con collation)	S1
contrasena	String	sí	—	Hash bcrypt (10 rondas). `select: false`: nunca se devuelve. RN-01 se valida en el DTO antes de hashear	S1
fechaNacimiento	Date	sí	—	Anterior a hoy (S-24)	S1
descripcion	String	no	''	trim, máx. 200 (S-06)	S1
imagenPerfilUrl	String	no	null	secure_url de Cloudinary (S-05)	S1
imagenPerfilPublicId	String	no	null	public_id de Cloudinary. No se expone en la API	S1
perfil	String (enum)	sí	'usuario'	enum ['usuario','administrador']. Solo un admin asigna 'administrador' (E-15)	S1
habilitado	Boolean	sí	true	Baja/alta lógica (E-16/E-17). Exigido en S4; conviene crearlo en S1	S4 (adelantar a S1)
createdAt / updatedAt	Date	auto	auto	timestamps de Mongoose	S1

4.2 Índices

Índice	Tipo	Para qué
`{ correo: 1 }` unique	Único	Unicidad de correo (409) y búsqueda en el login
`{ nombreUsuario: 1 }` unique, collation `{ locale: 'es', strength: 2 }`	Único	Unicidad sin distinguir mayúsculas («Ana.G» = «ana.g»)

4.3 Esquema Mongoose

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { HydratedDocument } from 'mongoose';

export type UsuarioDocument = HydratedDocument<Usuario>;
export const PERFILES = ['usuario', 'administrador'] as const;

@Schema({ collection: 'usuarios', timestamps: true })
export class Usuario {
  @Prop({ required: true, trim: true, minlength: 2, maxlength: 50 })
  nombre: string;

  @Prop({ required: true, trim: true, minlength: 2, maxlength: 50 })
  apellido: string;

  @Prop({ required: true, trim: true, lowercase: true, unique: true })
  correo: string;

  @Prop({ required: true, trim: true, minlength: 3, maxlength: 20, match: /^[a-zA-Z0-9._]+$/ })
  nombreUsuario: string;

  @Prop({ required: true, select: false })
  contrasena: string;

  @Prop({ required: true })
  fechaNacimiento: Date;

  @Prop({ trim: true, maxlength: 200, default: '' })
  descripcion: string;

  @Prop({ type: String, default: null })
  imagenPerfilUrl: string | null;

  @Prop({ type: String, default: null })
  imagenPerfilPublicId: string | null;

  @Prop({ type: String, enum: PERFILES, default: 'usuario' })
  perfil: 'usuario' | 'administrador';

  @Prop({ default: true })
  habilitado: boolean;
}

export const UsuarioSchema = SchemaFactory.createForClass(Usuario);
UsuarioSchema.index({ nombreUsuario: 1 },
  { unique: true, collation: { locale: 'es', strength: 2 } });
```

4.4 Ejemplo

```
{
  _id: ObjectId("6718a2f0c1e4b2a9d0f1a001"),
  nombre: "Ana", apellido: "Gómez",
  correo: "ana@mail.com", nombreUsuario: "ana.g",
  contrasena: "$2b$10$Qe3v1y0w8m6Jt0eS2vH8Q.7nq0bXo5vR3Yf1uJm2c9pW6s1aLk4Ze",
  fechaNacimiento: ISODate("2001-05-14T00:00:00.000Z"),
  descripcion: "Coleccionista de eurogames, siempre con un mazo de cartas en la mochila",
  imagenPerfilUrl:
"https://res.cloudinary.com/<cloud>/image/upload/v1/jugatelo/perfiles/ana.webp",
  imagenPerfilPublicId: "jugatelo/perfiles/ana",
  perfil: "usuario", habilitado: true,
  createdAt: ISODate("2026-10-20T13:02:11.000Z"), updatedAt: ISODate("2026-10-
20T13:02:11.000Z"), __v: 0
}
```

Notas - Para buscar por nombreUsuario usando el índice case-insensitive, la consulta debe llevar la misma collation: `findOne({ nombreUsuario }).collation({ locale: 'es', strength: 2 })`. Un E11000 (clave duplicada) al guardar se traduce a 409. - El login selecciona explícitamente el hash: `findOne({ correo }).select('+contrasena')` y luego `bcrypt.compare(contrasena, usuario.contrasena)`.

5. Colección `publicaciones`

5.1 Esquema

Campo	Tipo	Req.	Default	Validación / notas	Sprint
_id	ObjectId	sí	auto		S2
tipo	String (enum)	sí	—	enum ['juego','juntada'] (S-27). No se puede cambiar después de crear	S2
titulo	String	sí	—	trim, 3–100 (RN-11). Nombre del juego o título de la juntada	S2
descripcion	String	sí	—	trim, 1–1000 (RN-11, S-02)	S2
jugadoresMin	Number	no	null	Solo `juego`. Entero 1–20 (RN-14)	S2
jugadoresMax	Number	no	null	Solo `juego`. Entero 1–20, ≥ jugadoresMin (RN-14)	S2
zona	String	no*	null	Solo `juntada` (*obligatoria en ese tipo). trim, 2–80, texto libre (S-28)	S2
fechaJuntada	Date	no*	null	Solo `juntada` (*obligatoria). Posterior a la fecha de creación (RN-14)	S2
cupo	Number	no	null	Solo `juntada`. Entero 2–50, informativo (S-30)	S2
imagenUrl	String	no	null	secure_url de Cloudinary (S-03)	S2
imagenPublicId	String	no	null	public_id de Cloudinary. No se expone	S2
usuario	ObjectId	sí	—	ref 'Usuario'. Se toma del token, nunca del body	S2
meGusta	[ObjectId]	sí	[]	ref 'Usuario'. Sin repetidos (`$addToSet`). No se expone; la API devuelve `meGustaDado`	S2
cantidadMeGusta	Number	sí	0	min 0. Se actualiza con `$inc` junto con `meGusta`	S2
activa	Boolean	sí	true	Baja lógica (E-08). false oculta la publicación y sus comentarios (S-14)	S2

Campo	Tipo	Req.	Default	Validación / notas	Sprint
createdAt / updatedAt	Date	auto	auto	createdAt = fecha de la publicación (orden por fecha, estadísticas). Distinta de `fechaJuntada`, que es cuándo se juega	S2

5.2 Índices

Índice	Para qué
`{ activa: 1, createdAt: -1 }`	Listado por fecha (E-06 orden=fecha)
`{ activa: 1, cantidadMeGusta: -1, createdAt: -1 }`	Listado por me gusta (E-06 orden=meGusta), desempate por fecha
`{ activa: 1, tipo: 1, createdAt: -1 }`	Listado filtrado por tipo (E-06 tipo=juegoljuntada)
`{ activa: 1, tipo: 1, zona: 1 }`	Juntadas de una zona (E-06 tipo=juntada&zona=...)
`{ usuario: 1, activa: 1, createdAt: -1 }`	Filtro por usuario (E-06 usuarioId; Mi Perfil)
`{ createdAt: 1 }`	Rango de fechas de E-18 (publicaciones por usuario)

5.3 Esquema Mongoose

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { HydratedDocument, Types } from 'mongoose';

export type PublicacionDocument = HydratedDocument<Publicacion>;

export const TIPOS_PUBLICACION = ['juego', 'juntada'] as const;
export type TipoPublicacion = (typeof TIPOS_PUBLICACION)[number];

@Schema({ collection: 'publicaciones', timestamps: true })
export class Publicacion {
  @Prop({ type: String, enum: TIPOS_PUBLICACION, required: true, immutable: true })
  tipo: TipoPublicacion;

  @Prop({ required: true, trim: true, minlength: 3, maxlength: 100 })
  titulo: string;

  @Prop({ required: true, trim: true, minlength: 1, maxlength: 1000 })
  descripcion: string;

  @Prop({ type: Number, min: 1, max: 20, default: null })
  jugadoresMin: number | null;

  @Prop({ type: Number, min: 1, max: 20, default: null })
  jugadoresMax: number | null;

  @Prop({ type: String, trim: true, minlength: 2, maxlength: 80, default: null })
  zona: string | null;

  @Prop({ type: Date, default: null })
  fechaJuntada: Date | null;

  @Prop({ type: Number, min: 2, max: 50, default: null })
  cupo: number | null;

  @Prop({ type: String, default: null })
  imagenUrl: string | null;

  @Prop({ type: String, default: null })
  imagenPublicId: string | null;

  @Prop({ type: Types.ObjectId, ref: 'Usuario', required: true })
  usuario: Types.ObjectId;

  @Prop({ type: [{ type: Types.ObjectId, ref: 'Usuario' }], default: [] })
  meGusta: Types.ObjectId[];

  @Prop({ default: 0, min: 0 })
  cantidadMeGusta: number;
```

```
  @Prop({ default: true })
  activa: boolean;
}

export const PublicacionSchema = SchemaFactory.createForClass(Publicacion);
PublicacionSchema.index({ activa: 1, createdAt: -1 });
PublicacionSchema.index({ activa: 1, cantidadMeGusta: -1, createdAt: -1 });
PublicacionSchema.index({ activa: 1, tipo: 1, createdAt: -1 });
PublicacionSchema.index({ activa: 1, tipo: 1, zona: 1 });
PublicacionSchema.index({ usuario: 1, activa: 1, createdAt: -1 });
PublicacionSchema.index({ createdAt: 1 });
```

5.4 Ejemplo

```
{
  _id: ObjectId("6718b0aac1e4b2a9d0f1a0f3"),
  tipo: "juntada",
  titulo: "Tarde de Catan y Carcassonne",
  descripcion: "Llevo Catan y Carcassonne. Hay mate y snacks, traigan ganas de jugar.",
  jugadoresMin: null, jugadoresMax: null,
  zona: "Villa Crespo, CABA",
  fechaJuntada: ISODate("2026-11-08T20:00:00.000Z"),
  cupo: 6,
  imagenUrl:
"https://res.cloudinary.com/<cloud>/image/upload/v1/jugatelo/publicaciones/x1.webp",
  imagenPublicId: "jugatelo/publicaciones/x1",
  usuario: ObjectId("6718a2f0c1e4b2a9d0f1a001"),
  meGusta: [ ObjectId("6718a2f0c1e4b2a9d0f1a009"), ObjectId("6718a2f0c1e4b2a9d0f1a00c") ],
  cantidadMeGusta: 2, activa: true,
  createdAt: ISODate("2026-10-21T22:15:40.000Z"), updatedAt: ISODate("2026-10-
21T22:40:02.000Z"), __v: 0
}
```

Ejemplo de un documento `tipo: "juego"`: lleva `jugadoresMin: 3, jugadoresMax: 5` y `zona`, `fechaJuntada` y `cupo` en `null`.

Importante: `meGusta` y `cantidadMeGusta` siempre se modifican en la **misma** operación `updateOne` (ver 7.4) para que el contador nunca quede desfasado del array.

6. Colección `comentarios`

Colección propia porque se paginan, se editan y se cuentan por rango de fechas (cap. 3).

6.1 Esquema

Campo	Tipo	Req.	Default	Validación / notas	Sprint
_id	ObjectId	sí	auto		S2 (esquema, S-09) · S3
publicacion	ObjectId	sí	—	ref 'Publicacion'. Viene de la ruta (:publicacionId)	S2 · S3
usuario	ObjectId	sí	—	ref 'Usuario'. Se toma del token	S2 · S3

Campo	Tipo	Req.	Default	Validación / notas	Sprint
mensaje	String	sí	—	trim, 1–500 (RN-11)	S2 · S3
modificado	Boolean	sí	false	Pasa a true en el PUT (E-13); el front muestra «(editado)»	S3
createdAt / updatedAt	Date	auto	auto	createdAt = fecha del comentario (orden y estadísticas)	S2 · S3

6.2 Índices

Índice	Para qué
`{ publicacion: 1, createdAt: -1 }`	Listado paginado por publicación, más recientes primero (E-11); conteo por publicación; últimos 3 en Mi Perfil
`{ createdAt: 1 }`	Rango de fechas de E-19 y E-20

6.3 Esquema Mongoose

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { HydratedDocument, Types } from 'mongoose';

export type ComentarioDocument = HydratedDocument<Comentario>;

@Schema({ collection: 'comentarios', timestamps: true })
export class Comentario {
  @Prop({ type: Types.ObjectId, ref: 'Publicacion', required: true })
  publicacion: Types.ObjectId;

  @Prop({ type: Types.ObjectId, ref: 'Usuario', required: true })
  usuario: Types.ObjectId;

  @Prop({ required: true, trim: true, minlength: 1, maxlength: 500 })
  mensaje: string;

  @Prop({ default: false })
  modificado: boolean;
}

export const ComentarioSchema = SchemaFactory.createForClass(Comentario);
ComentarioSchema.index({ publicacion: 1, createdAt: -1 });
ComentarioSchema.index({ createdAt: 1 });
```

6.4 Ejemplo

```
{
  _id: ObjectId("6718c3d1c1e4b2a9d0f1a2b7"),
  publicacion: ObjectId("6718b0aac1e4b2a9d0f1a0f3"),
  usuario: ObjectId("6718a2f0c1e4b2a9d0f1a009"),
  mensaje: "Me sumo, llevo el Dixit",
  modificado: true,
  createdAt: ISODate("2026-10-21T22:30:00.000Z"), updatedAt: ISODate("2026-10-
21T22:41:13.000Z"), __v: 0
}
```

S-14: los comentarios no tienen campo de baja propio. Se ocultan cuando su publicación tiene `activa = false`, filtrando por la publicación.

7. Consultas y aggregations principales

`yo` es el ObjectId del usuario del token (decorador `@UsuarioActual()`). Los parámetros llegan validados por los DTOs.

7.1 Listado paginado con orden (E-06) — `PublicacionesService.listar()`

Excluye bajas lógicas, filtra opcionalmente por usuario, tipo y zona (S-28), ordena por fecha (default) o por me gusta, pagina con offset/limit y calcula `meGustaDado` para el usuario del token. El total se cuenta con el mismo filtro.

```
const filtro: FilterQuery<Publicacion> = { activa: true };
if (usuarioId) filtro.usuario = new Types.ObjectId(usuarioId);
if (q.tipo) filtro.tipo = q.tipo;
if (q.zona) filtro.zona = { $regex: escaparRegex(q.zona), $options: 'i' };

const orden = q.orden === 'meGusta'
  ? { cantidadMeGusta: -1, createdAt: -1, _id: -1 }
  : { createdAt: -1, _id: -1 };

const pipeline: PipelineStage[] = [
  { $match: filtro },
  { $sort: orden },
  { $skip: q.offset },   // def. 0
  { $limit: q.limit },   // def. 10, máx. 50
  { $lookup: { from: 'usuarios', localField: 'usuario', foreignField: '_id', as: 'usuario',
      pipeline: [{ $project: { nombreUsuario: 1, nombre: 1, apellido: 1, imagenPerfilUrl: 1 } }]
} },
  { $unwind: '$usuario' },
  { $addFields: { meGustaDado: { $in: [yo, '$meGusta'] } } },
  { $project: { meGusta: 0, imagenPublicId: 0, activa: 0, __v: 0 } },
];
if (q.comentarios > 0) pipeline.push(...etapaComentarios(q.comentarios)); // ver 7.3

const [datos, total] = await Promise.all([
  this.publicacionModel.aggregate(pipeline),
  this.publicacionModel.countDocuments(filtro),
]);
return { datos, total, offset: q.offset, limit: q.limit };
```

Nota: `$lookup` con localField/foreignField **y** pipeline requiere MongoDB ≥ 5.0 (Atlas lo cumple). Los índices compuestos del cap. 5 cubren `$match` + `$sort` de los dos órdenes.

7.2 Filtros por usuario, tipo y zona

Es E-06 con `usuarioId`, `tipo` y/o `zona`: el `$match` usa el índice `{ usuario: 1, activa: 1, createdAt: -1 }` (usuario) o `{ activa: 1, tipo: 1, ... }` (tipo y zona). Si usuarioId no es un ObjectId válido o tipo no es `juego` / `juntada`, el DTO responde 400. La zona se busca por coincidencia parcial sin distinguir mayúsculas; `escaparRegex` neutraliza los caracteres especiales para que lo escrito se tome literal. `GET /publicaciones?usuarioId=6718a2f0c1e4b2a9d0f1a001&offset=0&limit=10&orden=fecha` `GET /publicaciones?tipo=juntada&zona=crespo&orden=fecha`

7.3 Últimas 3 publicaciones con sus comentarios (Mi Perfil)

Mi Perfil llama a `GET /publicaciones?usuarioId=<yo>&limit=3&comentarios=3`. La etapa extra trae los 3 comentarios más recientes de cada publicación con su autor (S-12).

```
const etapaComentarios = (n: number): PipelineStage[] => [
  { $lookup: {
      from: 'comentarios',
      let: { pid: '$_id' },
      pipeline: [
        { $match: { $expr: { $eq: ['$publicacion', '$$pid'] } } },
        { $sort: { createdAt: -1 } },
        { $limit: n },
        { $lookup: { from: 'usuarios', localField: 'usuario', foreignField: '_id', as:
'usuario',
            pipeline: [{ $project: { nombreUsuario: 1, imagenPerfilUrl: 1 } }] } },
        { $unwind: '$usuario' },
        { $project: { __v: 0 } },
      ],
      as: 'comentarios',
  } },
];
```

7.4 Dar y quitar «Me gusta» (E-09 / E-10)

Operaciones atómicas sobre un solo documento: la condición del filtro garantiza un único me gusta por usuario y que el contador se mueva junto con el array, aun con dos clics simultáneos.

```
// E-09 dar me gusta
const r = await this.publicacionModel.updateOne(
  { _id: id, activa: true, meGusta: { $ne: yo } },
  { $addToSet: { meGusta: yo }, $inc: { cantidadMeGusta: 1 } },
);
if (r.modifiedCount === 0) {
  const existe = await this.publicacionModel.exists({ _id: id, activa: true });
  if (!existe) throw new NotFoundException('La publicación no existe');
  throw new ConflictException('Ya diste me gusta a esta publicación'); // 409
}

// E-10 quitar me gusta
const r2 = await this.publicacionModel.updateOne(
  { _id: id, activa: true, meGusta: yo },
  { $pull: { meGusta: yo }, $inc: { cantidadMeGusta: -1 } },
);
if (r2.modifiedCount === 0) throw new NotFoundException('No habías dado me gusta a esta
publicación'); // S-13
```

7.5 Detalle y baja lógica (E-07 / E-08)

```
// E-07 detalle: mismo pipeline que 7.1 con $match { _id: id, activa: true } y $limit 1 -> 404
si vacío

// E-08 baja lógica: solo autor o administrador
const pub = await this.publicacionModel.findOne({ _id: id, activa: true }, { usuario: 1 });
if (!pub) throw new NotFoundException('La publicación no existe');
const esAutor = pub.usuario.equals(yo);
if (!esAutor && usuario.perfil !== 'administrador')
  throw new ForbiddenException('No tenés permiso para eliminar esta publicación');
await this.publicacionModel.updateOne({ _id: id }, { $set: { activa: false } });
// los comentarios quedan ocultos porque E-11/E-12/E-13 exigen publicación activa (S-14)
```

7.6 Comentarios paginados, alta y edición (E-11 / E-12 / E-13)

```
// E-11 listar: más recientes primero
await this.asegurarPublicacionActiva(publicacionId); // 404 si no existe o activa = false
const filtro = { publicacion: new Types.ObjectId(publicacionId) };
const [datos, total] = await Promise.all([
  this.comentarioModel.find(filtro)
    .sort({ createdAt: -1, _id: -1 })
    .skip(offset).limit(limit)   // def. 0 / 5
    .populate('usuario', 'nombreUsuario imagenPerfilUrl')
    .select('-__v').lean(),
  this.comentarioModel.countDocuments(filtro),
]);

// E-12 agregar
await this.asegurarPublicacionActiva(publicacionId);
const nuevo = await this.comentarioModel.create({ publicacion: publicacionId, usuario: yo,
mensaje });
// 201 -> se devuelve populado con el autor

// E-13 editar: solo el autor
const com = await this.comentarioModel.findOne({ _id: comentarioId, publicacion: publicacionId
});
if (!com) throw new NotFoundException('El comentario no existe');
await this.asegurarPublicacionActiva(publicacionId);
if (!com.usuario.equals(yo)) throw new ForbiddenException('Solo el autor puede editar el
comentario');
com.mensaje = mensaje;
com.modificado = true;
await com.save(); // updatedAt se actualiza solo
```

Nota: con paginación por offset, si alguien comenta mientras otro usuario presiona «Cargar más», el bloque siguiente puede traer un comentario ya mostrado. El front descarta repetidos por `_id`.

7.7 Estadísticas por rango de fechas (S4)

Los tres endpoints reciben `desde` y `hasta` (AAAA-MM-DD). `RangoFechasQueryDto` los convierte a un rango inclusivo en hora de Argentina (S-18) y valida desde ≤ hasta (400). Solo se cuenta contenido de publicaciones activas (S-14).

```
const desde = new Date(`${q.desde}T00:00:00.000-03:00`);
const hasta = new Date(`${q.hasta}T23:59:59.999-03:00`);
const rango = { createdAt: { $gte: desde, $lte: hasta } };
```

E-18 · Publicaciones por usuario (barras)

```
this.publicacionModel.aggregate([
  { $match: { activa: true, ...rango } },
  { $group: { _id: '$usuario', cantidad: { $sum: 1 } } },
  { $lookup: { from: 'usuarios', localField: '_id', foreignField: '_id', as: 'u',
      pipeline: [{ $project: { nombreUsuario: 1 } }] } },
  { $unwind: '$u' },
  { $project: { _id: 0, usuarioId: '$_id', nombreUsuario: '$u.nombreUsuario', cantidad: 1 } },
  { $sort: { cantidad: -1, nombreUsuario: 1 } },
]);
// -> [ { usuarioId, nombreUsuario: "ana.g", cantidad: 7 }, ... ]
```

E-19 · Comentarios realizados en el lapso (total + líneas por día)

```
this.comentarioModel.aggregate([
  { $match: rango },
  { $lookup: { from: 'publicaciones', localField: 'publicacion', foreignField: '_id', as: 'p',
      pipeline: [{ $match: { activa: true } }, { $project: { _id: 1 } }] } },
  { $match: { 'p.0': { $exists: true } } }, // descarta publicaciones dadas de baja
  { $facet: {
      total: [{ $count: 'cantidad' }],
      porDia: [
        { $group: { _id: { $dateToString: { format: '%Y-%m-%d', date: '$createdAt',
                                            timezone: 'America/Argentina/Buenos_Aires' } },
                    cantidad: { $sum: 1 } } },
        { $sort: { _id: 1 } },
        { $project: { _id: 0, fecha: '$_id', cantidad: 1 } },
      ],
  } },
]);
// -> { total: 58, porDia: [ { fecha: "2026-10-20", cantidad: 9 }, ... ] }
// $facet devuelve total como array: el servicio lo aplana a número; los días sin datos los
completa el front
```

E-20 · Comentarios en cada publicación en el lapso (torta / dona)

```
this.comentarioModel.aggregate([
  { $match: rango },
  { $group: { _id: '$publicacion', cantidad: { $sum: 1 } } },
  { $lookup: { from: 'publicaciones', localField: '_id', foreignField: '_id', as: 'p',
      pipeline: [{ $match: { activa: true } }, { $project: { titulo: 1 } }] } },
  { $unwind: '$p' },
  { $sort: { cantidad: -1 } },
  { $limit: 10 }, // top 10 (S-18)
  { $project: { _id: 0, publicacionId: '$_id', titulo: '$p.titulo', cantidad: 1 } },
]);
// -> [ { publicacionId, titulo: "Nunca le dije a nadie", cantidad: 14 }, ... ]
```

Supuesto: «comentarios en cada publicación en un lapso» = comentarios creados dentro del lapso, agrupados por publicación (pregunta al docente).

7.8 Usuarios: listado, alta y bajas lógicas (E-14 a E-17)

```
// E-14 listado (sin contrasena por select: false)
this.usuarioModel.find().select('-imagenPerfilPublicId -__v').sort({ createdAt: -1 }).lean();

// E-16 deshabilitar / E-17 habilitar
if (id === yo.toString()) throw new BadRequestException('No podés deshabilitarte a vos mismo');
// S-17
const u = await this.usuarioModel.findByIdAndUpdate(id, { $set: { habilitado: false } }, { new:
true });
if (!u) throw new NotFoundException('El usuario no existe');

// login y JwtAuthGuard verifican habilitado -> 403 si es false (RN-07, S-16)
```

8. Evolución del modelo por sprint

Sprint	usuarios	publicaciones	comentarios
S1	Todos los campos + índices únicos. `habilitado` se crea ya (default true) aunque se use en S4.	Módulo creado (esquema vacío o borrador).	—
S2	Sin cambios.	Todos los campos e índices: tipo, titulo, descripcion, jugadoresMin, jugadoresMax, zona, fechaJuntada, cupo, imagenUrl, imagenPublicId, usuario, meGusta, cantidadMeGusta, activa.	Esquema creado (publicacion, usuario, mensaje) para el `$lookup` de Mi Perfil (S-09).
S3	Sin cambios.	Sin cambios.	`modificado` + índice `{ publicacion: 1, createdAt: -1 }`; empiezan a crearse documentos (E-12).
S4	Se usa `habilitado` (E-16/E-17) y perfil «administrador» (E-15).	`activa` también la cambia el admin. Índice `{ createdAt: 1 }` para estadísticas.	Índice `{ createdAt: 1 }` para estadísticas.

8.1 Configuración de MongoDB Atlas

- Cluster gratuito; base de datos `jugatelo`. Usuario de BD con permisos de lectura/escritura solo sobre esa base.
- Network Access: el hosting gratuito del back suele no tener IP fija, por lo que normalmente se habilita `0.0.0.0/0`; compensarlo con contraseña fuerte y la URI solo en variables de entorno.
- Variable de entorno `MONGODB_URI` leída con ConfigModule. Nunca se sube al repositorio (`.env` en `.gitignore`).
- Los índices se crean con autoIndex en desarrollo; antes de la entrega verificar en Atlas (pestaña Indexes) que existan los únicos de usuarios, porque sin ellos el 409 no se dispara.

9. Anexo — Nomenclatura común

Idéntica a la del Doc 1 (sección 9): ver 1_analisis_funcional.md.